

THREAD PRIORITIES

21 February 2025 21:21

- Thread Priority in Java decides **which thread gets more CPU time** compared to others. **Higher priority threads get preference, but not a guarantee of execution order.**
- Use thread priority only as a hint, not a strict rule.
- JVM provide priority to each thread and according to these priorities JVM will allocate the processors .

Basics of Thread Priority

- Priority are represented in the form of integer value which ranges 1 to 10
1. Java assigns **default priority = 5** to all threads.
 2. Thread priority range:
 - **MIN_PRIORITY = 1** (Lowest)
 - **NORM_PRIORITY = 5** (Default)
 - **MAX_PRIORITY = 10** (Highest)
 3. **Higher-priority threads may execute first, but CPU scheduling is not guaranteed.**

Methods:

1. `setPriority()`
2. `getPriority()`

Point to be remembered:

1. Main thread priority is 5:
2. Window does not support priorities.

Real-Life Example: Priority in a Hospital 🏥

Hospital System	Java Thread Priority
Emergency patients get treated first	High Priority (10)
Regular checkup patients wait in line	Normal Priority (5)
General inquiries take more time	Low Priority (1)

CODE:

```
package multithreading;
class MyThread extends Thread {
    public MyThread(String name) {
        super(name);
    }

    public void run() {
        System.out.println(getName() + " is running with priority " + getPriority());
    }
}

public class ThreadPriorityExample {
    public static void main(String[] args) {
        // Creating three threads
        MyThread t1 = new MyThread("Low Priority Thread");
        MyThread t2 = new MyThread("Normal Priority Thread");
        MyThread t3 = new MyThread("High Priority Thread");

        // Setting priorities
        t1.setPriority(Thread.MIN_PRIORITY); // Priority 1
        t2.setPriority(Thread.NORM_PRIORITY); // Priority 5 (Default)
        t3.setPriority(Thread.MAX_PRIORITY); // Priority 10

        // Starting threads
        t1.start();
        t2.start();
        t3.start();
    }
}
```

➤ When is Thread Priority Useful?

1. **When Multiple Threads Are Competing for CPU Time:**
 - If the system is under **heavy load**, the CPU gives more preference to higher-priority threads.
 - Example: A **real-time data processing thread** (priority 8) vs. a **background logging thread** (priority 2).
2. **In Time-Sensitive Applications:**
 - Example: In a **media player**, a thread playing video should have **higher priority** than a thread writing logs.
3. **When Threads Have Different Levels of Importance:**

- Example:
 - A **high-priority thread (10)** may be handling **critical banking transactions**.
 - A **low-priority thread (2)** may be **sending email notifications**.

➤ **Why Thread Priority is NOT Reliable?**

1. JVM's Thread Scheduler Behavior is Unpredictable:

- The JVM does not always follow priority strictly.
- Different JVM implementations may schedule threads differently.

2. Operating System Dependent:

- Some OS thread schedulers **ignore Java thread priority**.
- Example: **Linux systems use time-sharing scheduling**, so priority may have little effect.

3. If All Threads Are Running, Priority May Not Matter:

- If no thread is waiting, the OS might **schedule threads randomly**.